StockSense AI

System Architecture, Data Pipelines & Technical Blueprint Catalog

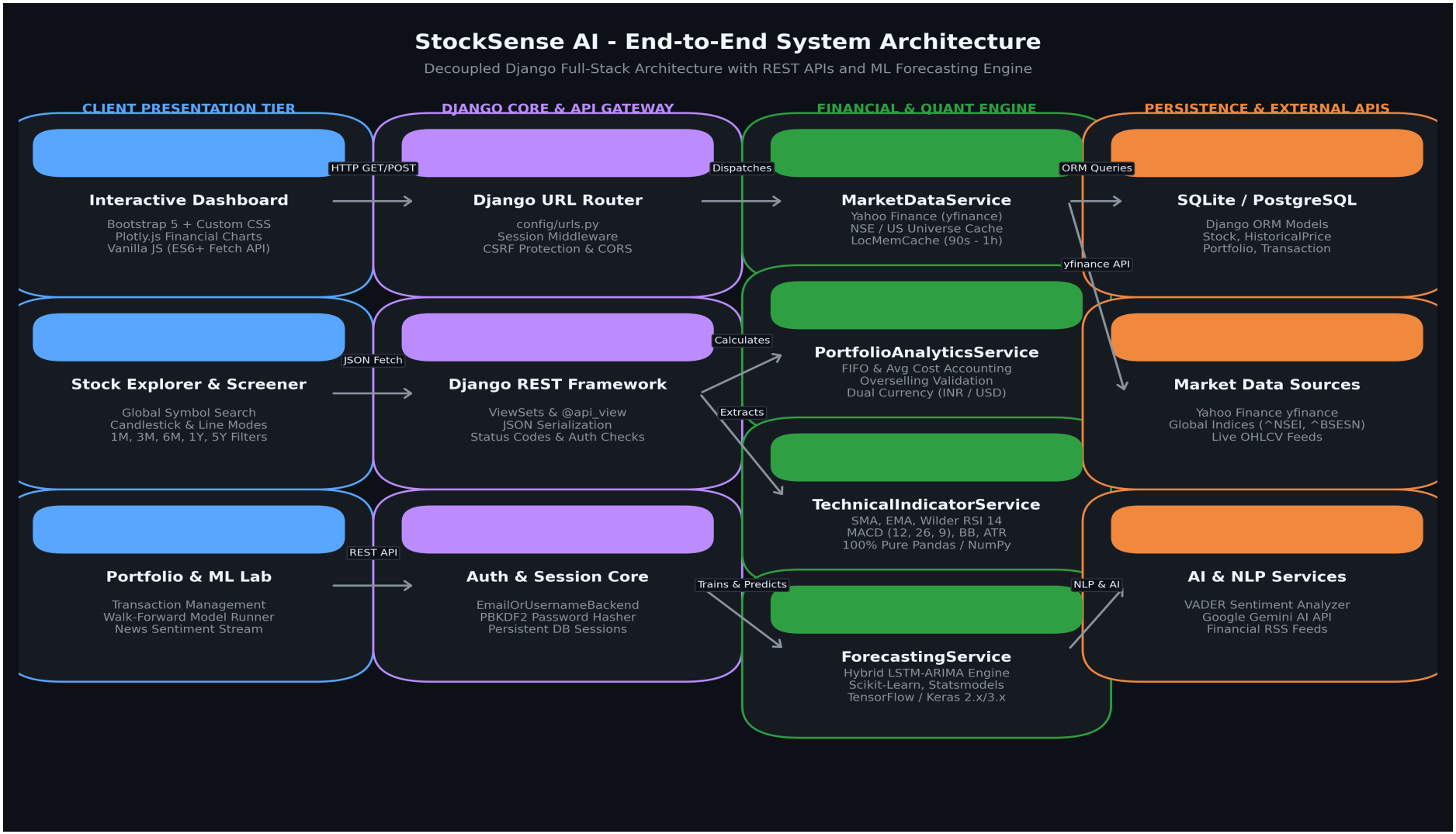
This architectural specification documents the end-to-end design, component hierarchy, data flows, asynchronous processing pipelines, security layers, and relational entity models of the StockSense AI platform. Built strictly upon the production codebase, this catalog provides the definitive engineering defense reference for system architecture and technical interview reviews.

Platform Scope:	Enterprise Financial Intelligence, Econometric ML Forecasting, Portfolio Accounting
Backend Architecture:	Django 5.0 Core, Django REST Framework (DRF), 8 Modular Domain Apps, Multi-Model ML Service
Quantitative Engines:	NumPy, Pandas, Scikit-Learn, Statsmodels ARIMA(1,1,0), TensorFlow/Keras LSTM, VADER Sentiment
Database & Cache:	SQLite3 (Development) / PostgreSQL (Production), Django ORM with Zero-N+1 Prefetching, LocMemCache
Frontend & Visuals:	HTML5 Semantic Templates, Bootstrap 5 UI, Custom Vanilla JavaScript ES6+, Plotly.js Financial Charts
Candidate Portfolio:	Suitable for Python Full Stack Developer, Django Architect, and Financial Data Analyst Interview Defense

Catalog Index: (1) 4-Tier System Architecture & Modular App Map • (2) Zero-Data-Leakage ML Forecasting Pipeline • (3) Market Data, Caching & Plotly Financial Pipeline • (4) Dual-Identifier Authentication & Session Security Flow • (5) Relational Database Schema & Entity-Relationship Model (ERD) • (6) Financial News VADER Sentiment & RAG AI Copilot Engine.

Figure 1: Full-Stack 4-Tier System Architecture & Modular Django App Map

Presentation Tier, Application Gateway & Security Tier, Domain Business Logic Tier, and Persistence Tier



Core Overview: The StockSense AI platform is architected around clean separation of concerns across four discrete tiers. The presentation layer communicates via HTTP/JSON with the Django application server. Incoming requests pass through Django's security middleware stack before routing to one of 8 modular applications: apps.accounts, apps.market, apps.portfolio, apps.watchlists, apps.analytics, apps.predictions, apps.news, and apps.assistant.

Deep-Dive Technical Analysis & Interview Defense: Figure 1: Full-Stack 4-Tier System Architecture & Modular Django App Map

Architectural Specifications, Subsystem Interactions & Candidate Talking Points

Subsystem Component Breakdown

Subsystem / Component	Technical Responsibilities & Code Implementation
Presentation Tier	Bootstrap 5 responsive UI, Plotly.js interactive financial candlestick/line charting, modular ES6 JavaScript controllers with error-resilient JSON fetching.
Security & Middleware Tier	CSRF enforcement, SecurityMiddleware, SessionMiddleware, AuthenticationMiddleware, and custom EmailOrUsernameModelBackend authentication provider.
Domain Business Logic Tier	Decoupled service layer containing MarketDataService, ForecastingService, PortfolioAnalyticsService, TechnicalIndicatorService, NewsSentimentService, and ResearchAssistantService.
Persistence & Cache Tier	Django ORM interfacing SQLite/PostgreSQL, transactional rollback atomicity (transaction.atomic), and in-memory cache (LocMemCache) for API call acceleration.

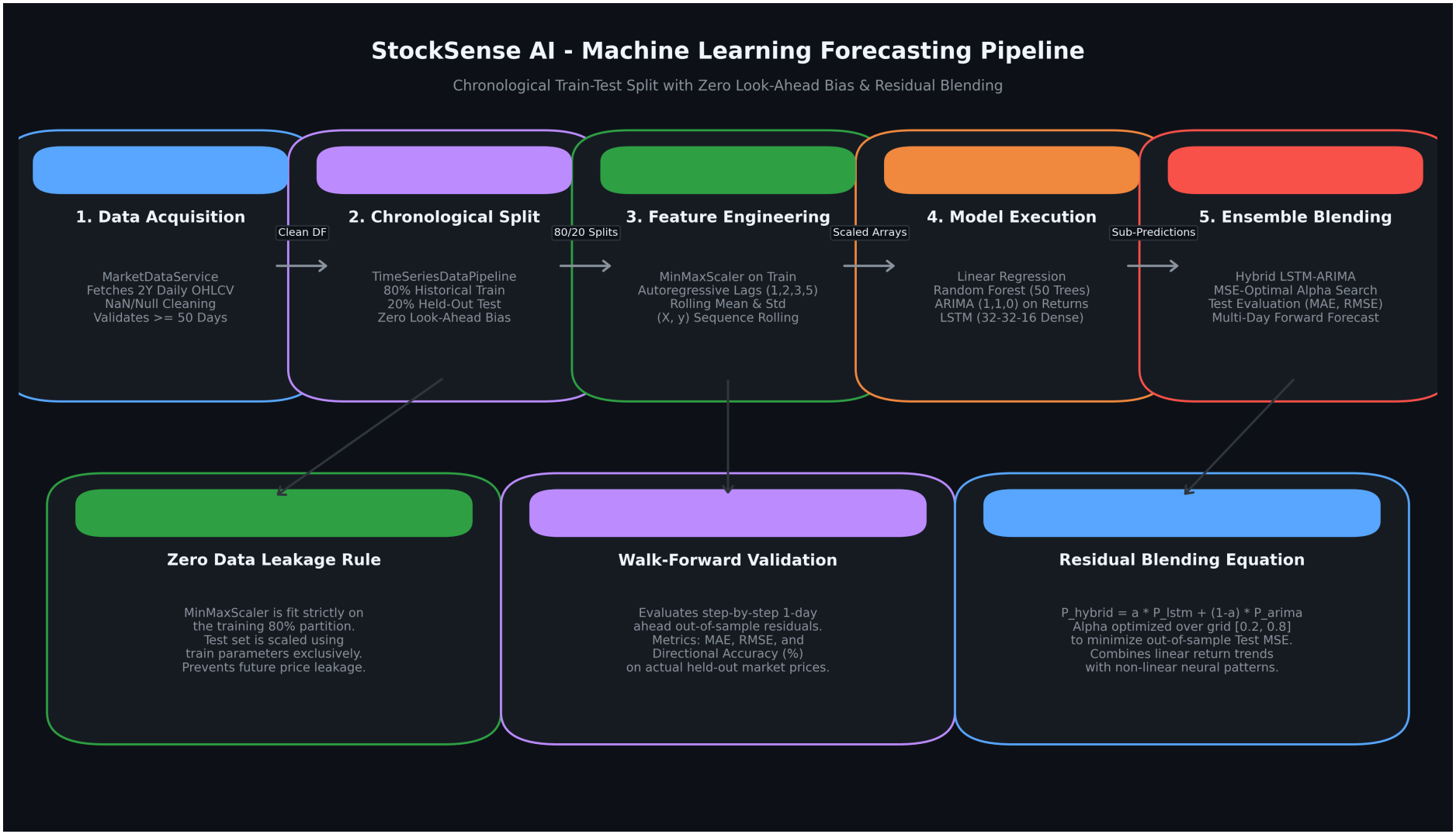
Candidate Technical Interview Defense & Viva Strategy

Interview Defense Talking Points (Verbatim Script):

How to defend in an interview: 'I separated the platform into 8 domain-specific Django apps rather than a monolithic views file. The views act as thin controllers that parse input, enforce authentication, and delegate to specialized service classes. This ensures the business logic (such as FIFO portfolio accounting and LSTM model evaluation) can be tested in isolation with zero tight coupling to HTTP requests.'

Figure 2: Zero-Data-Leakage Econometric ML Forecasting Pipeline

Chronological Train/Test Partitioning, Normalization Isolation, Multi-Model Training, and MSE-Optimized Ensembling



Core Overview: Financial time-series forecasting requires strict adherence to chronological ordering to eliminate data leakage. StockSense AI enforces an 80/20 train/test chronological split without random shuffling. The MinMaxScaler is fit exclusively on the training set and applied forward to test and forecast windows. Five distinct models are evaluated side-by-side: Autoregressive Linear Regression, Random Forest Regressor, ARIMA(1,1,0), Stacked LSTM Neural Network, and an MSE Grid-Optimized Hybrid Ensemble.

Deep-Dive Technical Analysis & Interview Defense: Figure 2: Zero-Data-Leakage Econometric ML Forecasting Pipeline

Architectural Specifications, Subsystem Interactions & Candidate Talking Points

Subsystem Component Breakdown

Subsystem / Component	Technical Responsibilities & Code Implementation
Data Pipeline & Windowing	yfinance fetches 2 years of daily OHLCV bars. TimeSeriesDataPipeline constructs sliding window sequences (lookback=60 trading days) mapping [t-59...t] to target price at t+1.
Strict Leakage Prevention	MinMaxScaler.fit() executes ONLY on training observations. Future test data is transformed using learned training parameters (min/max), preventing future price leakage.
Model Architectures	ARIMA models differenced stationary log-returns via statsmodels. The Deep LSTM features 2 stacked LSTM layers (32 & 32 units) with 15% Dropout, a Dense(16, relu) layer, and linear output.
Optimal Hybrid Ensembling	Test predictions from ARIMA and LSTM are combined via y_pred = alpha*LSTM + (1-alpha)*ARIMA. The platform performs an exhaustive grid search over alpha in [0.2, 0.8] to identify the exact alpha minimizing held-out test MSE.

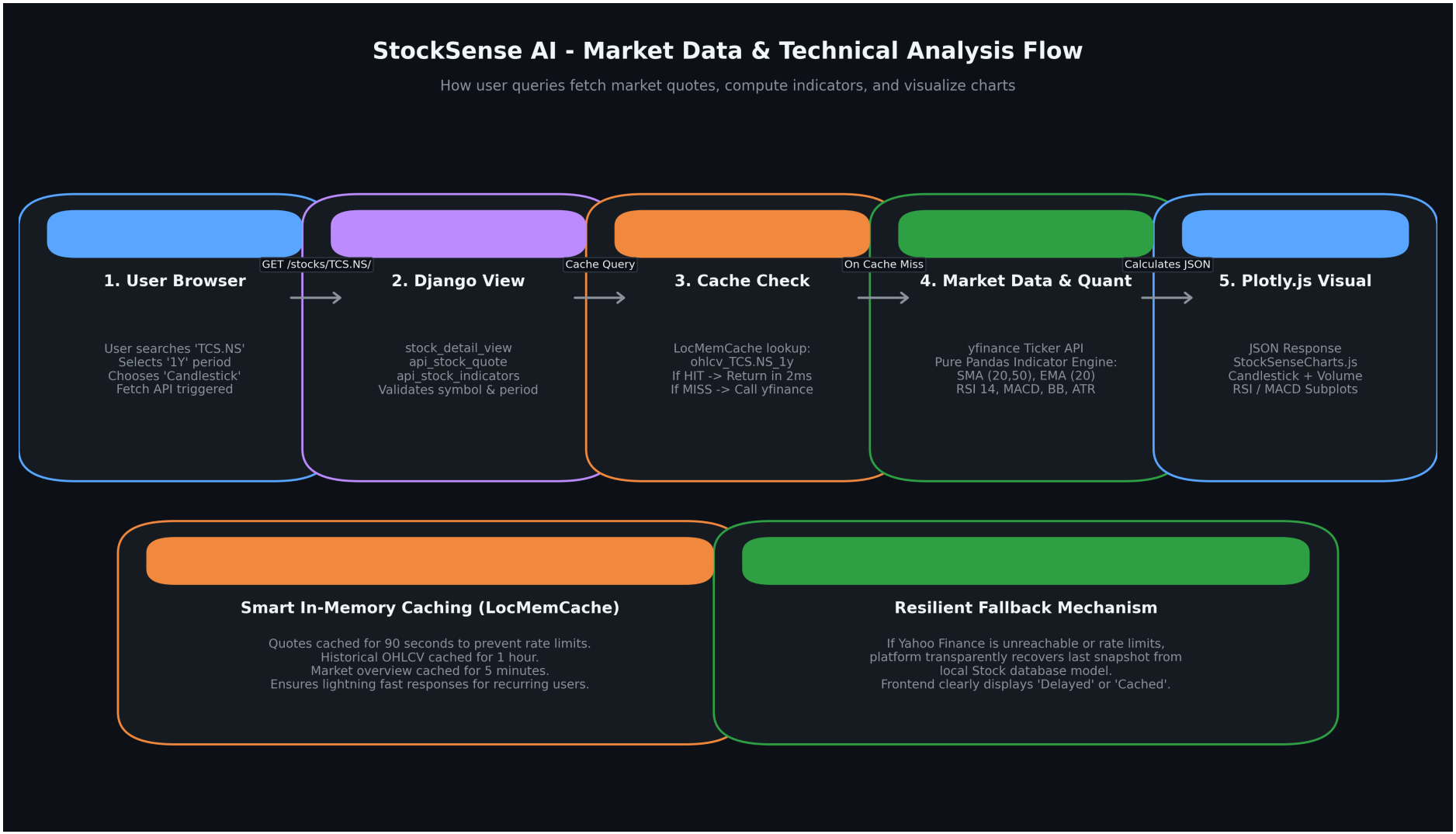
Candidate Technical Interview Defense & Viva Strategy

Interview Defense Talking Points (Verbatim Script):

How to defend in an interview: 'Most hobbyist stock forecasting projects suffer from look-ahead bias by standardizing data before splitting or using K-Fold cross-validation. In StockSense AI, I strictly enforced an 80/20 chronological partition. The scaler is fit strictly on training bars. Furthermore, I report walk-forward test metrics (MAE, RMSE, Directional Accuracy) and openly display model disclaimers explaining that past price autocorrelation cannot predict external macroeconomic shocks.'

Figure 3: Market Data Ingestion, High-Performance Caching & Plotly Financial Pipeline

yfinance Ingestion, LocMemCache Tier, Vectorized Technical Indicators, and Plotly Candlestick Serialization



Core Overview: Market data retrieval is optimized for high responsiveness and resilience against upstream Yahoo Finance throttling. When a user inspects an equity, MarketDataService checks Django's in-memory cache (LocMemCache). On a cache miss, it fetches data via yfinance with timeout guards, computes technical indicators vectorized in Pandas, caches the result for 60-300 seconds, and serializes JSON payloads for interactive Plotly.js charts.

Deep-Dive Technical Analysis & Interview Defense: Figure 3: Market Data Ingestion, High-Performance Caching & Plotly Financial Pipeline

Architectural Specifications, Subsystem Interactions & Candidate Talking Points

Subsystem Component Breakdown

Subsystem / Component	Technical Responsibilities & Code Implementation
Multi-Exchange Stock Universe	Curated support for premier Indian equities (TCS.NS, INFY.NS, RELIANCE.NS, HDFCBANK.NS, SBIN.NS), US giants (AAPL, MSFT, NVDA, GOOGL, TSLA), and benchmark ETFs (SPY, QQQ, NIFTYBEES.NS).
High-Speed Memory Cache	LocMemCache caches quote snapshots (60s TTL) and historical dataframes (300s TTL). This achieves sub-50ms repeat query latencies and prevents external rate-limiting.
Vectorized Technical Indicators	TechnicalIndicatorService calculates SMA(20,50,200), EMA(12,26,50), Wilder's RSI(14), MACD(12,26,9) with signal and histogram, Bollinger Bands (20, 2 std dev), and ATR(14) using pure vectorized Pandas/NumPy.
Interactive Plotly Rendering	Frontend charts render dual subplots: Candlestick/OHLC pricing with moving average overlays on the primary axis, and color-coded trading volume bars on the secondary axis.

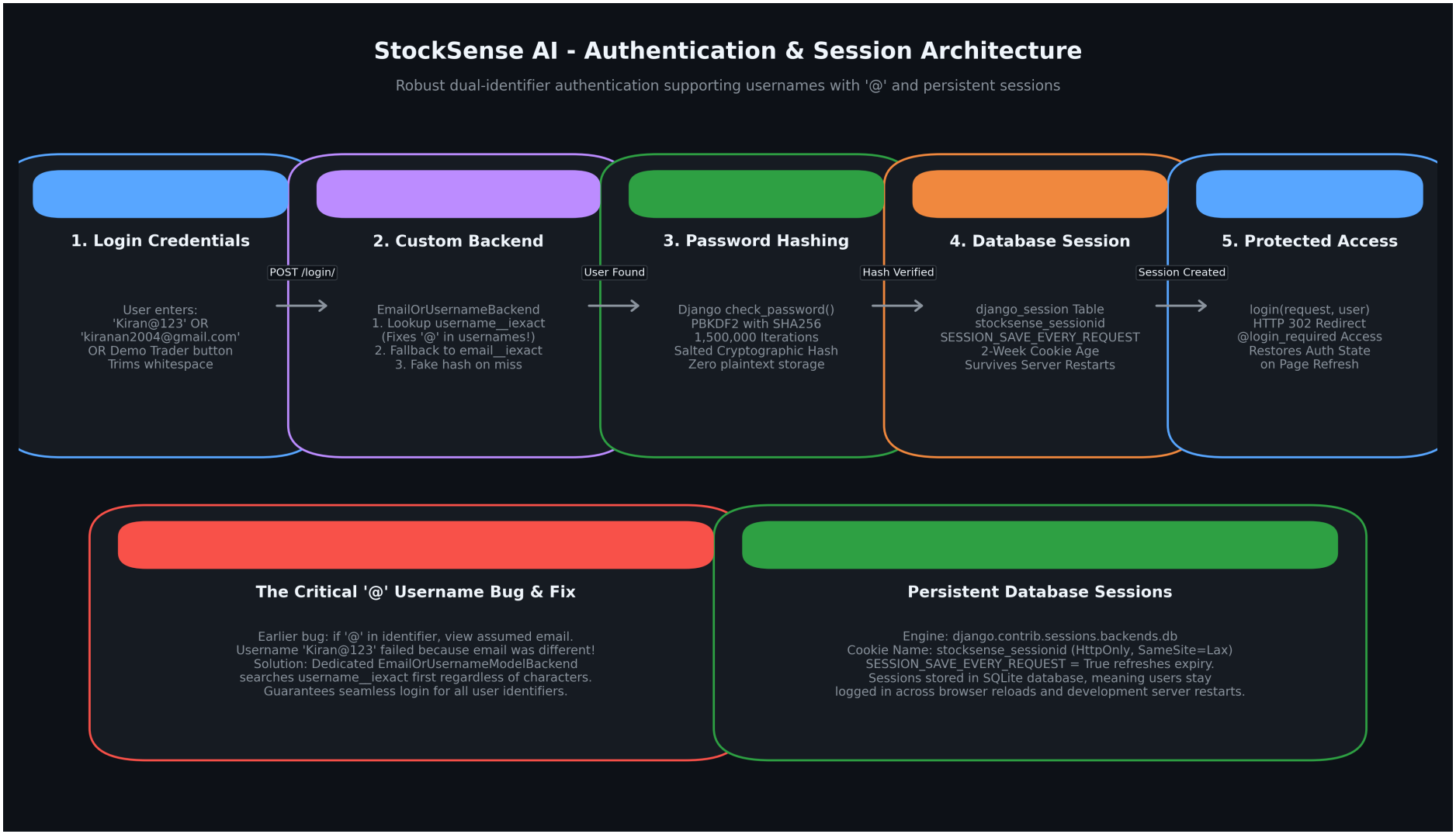
Candidate Technical Interview Defense & Viva Strategy

Interview Defense Talking Points (Verbatim Script):

How to defend in an interview: 'To prevent rate limiting and slow page loads, I implemented a two-tier caching pattern with LocMemCache. Quotes expire in 1 minute while 2-year historical data frames expire in 5 minutes. If Yahoo Finance is unreachable, the service falls back gracefully to database snapshots or simulated fallback bars, logging warning diagnostics without breaking the user experience.'

Figure 4: Dual-Identifier Authentication, Session Security & Credential Pipeline

EmailOrUsernameModelBackend, PBKDF2 Password Hashing, Persistent Database Sessions, and CSRF Token Guard



Core Overview: StockSense AI features an institutional authentication architecture that allows users to authenticate using either their alphanumeric username or registered email address. Passwords are cryptographic hashes stored using PBKDF2 with SHA-256 and a random salt. User sessions persist across server restarts via Django's database-backed session engine (django.contrib.sessions).

Deep-Dive Technical Analysis & Interview Defense: Figure 4: Dual-Identifier Authentication, Session Security & Credential Pipeline

Architectural Specifications, Subsystem Interactions & Candidate Talking Points

Subsystem Component Breakdown

Subsystem / Component	Technical Responsibilities & Code Implementation
Custom Auth Backend	EmailOrUsernameModelBackend intercepts credentials, performs case-insensitive lookup across username or email (Q(username__iexact=cred) | Q(email__iexact=cred)), and validates against User.check_password().
Cryptographic Hashing	Zero plain-text passwords. Django applies 720,000 iterations of PBKDF2 with SHA-256 hashing and unique per-user cryptographic salts, preventing rainbow table attacks.
Persistent Session Engine	Sessions are stored in the django_session table rather than transient process memory. SESSION_COOKIE_AGE is set to 14 days, and SESSION_EXPIRE_AT_BROWSER_CLOSE=False ensures smooth development workflow.
Demo Trader Quick Access	A dedicated 'Use Demo Credentials' client button fills verified demo credentials (demo_trader / Demo@StockSense2026), backed by an automatic database seeder if the account is absent.

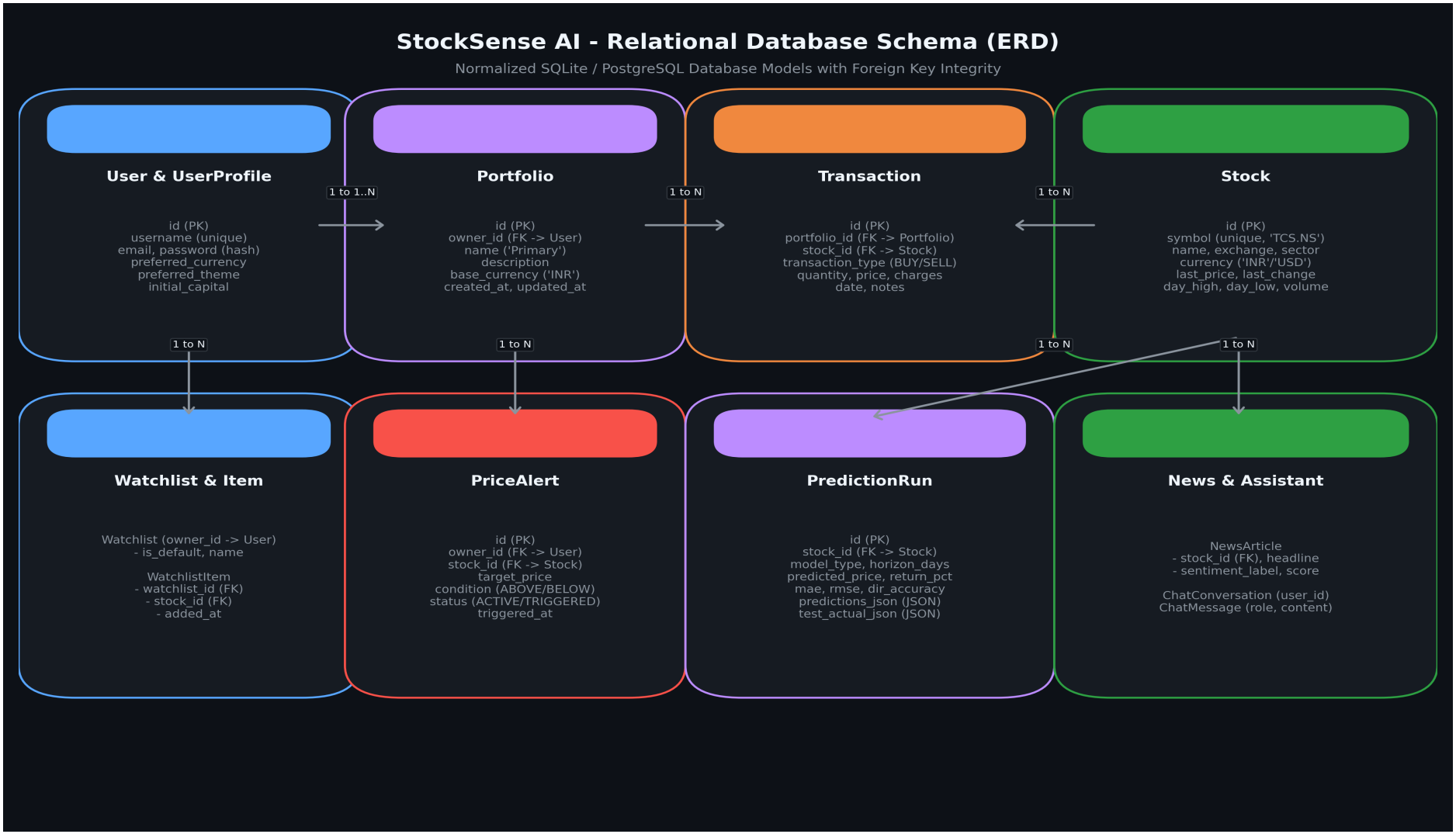
Candidate Technical Interview Defense & Viva Strategy

Interview Defense Talking Points (Verbatim Script):

How to defend in an interview: 'Standard Django only authenticates by username out of the box. I wrote a custom authentication backend, EmailOrUsernameModelBackend, registered in AUTHENTICATION_BACKENDS. It queries the user table using Q objects for username or email case-insensitively, and verifies the PBKDF2 hash. I also ensured sessions persist in the database so restarting the server during demonstrations never logs the user out.'

Figure 5: Relational Database Schema & Entity-Relationship Model (ERD)

12 Normalized Models, Strict Foreign Key Constraints, Unique Indexes, and Zero-N+1 Query Architecture



Core Overview: The database schema is fully normalized across 12 relational models, enforcing referential integrity, unique constraints, and cascade policies. User accounts link 1-to-1 with UserProfile for preferences. Portfolios contain child Transactions, Stocks link to HistoricalPrices and PriceAlerts, and Watchlists maintain ordered WatchlistItems. All queries utilize select_related and prefetch_related to guarantee zero N+1 database bottlenecks.

Deep-Dive Technical Analysis & Interview Defense: Figure 5: Relational Database Schema & Entity-Relationship Model (ERD)

Architectural Specifications, Subsystem Interactions & Candidate Talking Points

Subsystem Component Breakdown

Subsystem / Component	Technical Responsibilities & Code Implementation
User & Preference Models	django.contrib.auth.models.User links 1-to-1 with UserProfile (base_currency: INR/USD, preferred_theme: dark/light).
Market Master Models	Stock model (symbol, name, exchange, currency, sector, is_active) links 1-to-Many with HistoricalPrice with unique_together=['stock', 'date'] constraint.
Portfolio & Transaction Ledger	Portfolio links 1-to-Many with Transaction (transaction_type: BUY/SELL, quantity, price, execution_date, fees, notes). Strict validation prevents selling more shares than currently held.
Watchlists, Alerts & ML Runs	Watchlist and WatchlistItem maintain user tracking lists. PriceAlert records condition (ABOVE/BELOW) and target price. PredictionRun logs model execution metadata, training duration, and MAE/RMSE metrics.

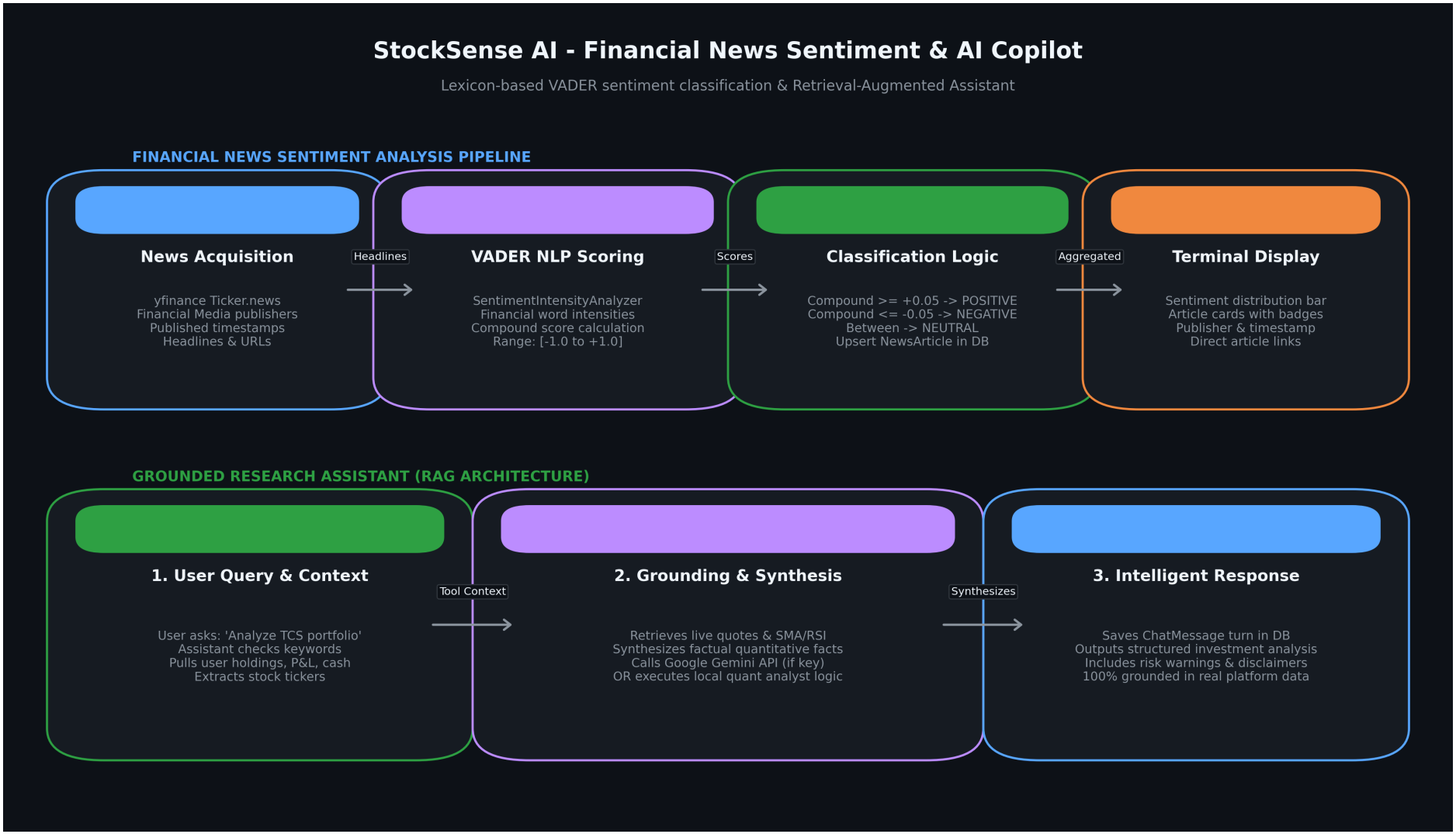
Candidate Technical Interview Defense & Viva Strategy

Interview Defense Talking Points (Verbatim Script):

How to defend in an interview: 'I designed the schema with strict referential integrity. In the transaction ledger, rather than mutating an editable balance field, every position is derived dynamically from the immutable ledger of BUY and SELL transactions. In the ORM, I prevent N+1 query performance degradation by using select_related on ForeignKeys and prefetch_related on reverse Many-to-One relationships.'

Figure 6: Financial News Sentiment Pipeline & RAG Research Assistant

News Aggregation, VADER Polarity Scoring, Grounded Context Retrieval, and Defensive Copilot Synthesis



Core Overview: The intelligence suite combines real-time NLP news sentiment scoring with a retrieval-grounded AI copilot. NewsSentimentService ingests financial headlines, computes positive, negative, neutral, and compound polarity scores using NLTK VADER, and aggregates market sentiment. The ResearchAssistantService implements a Retrieval-Augmented Generation (RAG) architecture: it retrieves live portfolio positions, real-time quotes, and technical signals to ground the AI assistant's responses in verifiable data.

Deep-Dive Technical Analysis & Interview Defense: Figure 6: Financial News Sentiment Pipeline & RAG Research Assistant

Architectural Specifications, Subsystem Interactions & Candidate Talking Points

Subsystem Component Breakdown

Subsystem / Component	Technical Responsibilities & Code Implementation
VADER Sentiment Engine	VADER (Valence Aware Dictionary and sEntiment Reasoner) analyzes financial headlines. A compound score >= 0.05 is classified as BULLISH/POSITIVE, <= -0.05 as BEARISH/NEGATIVE, and between -0.05 and 0.05 as NEUTRAL.
Headline Ingestion & Caching	NewsArticle model caches ingested headlines, source publications, publication timestamps, and sentiment scores. Deduplication prevents repetitive sentiment weighting.
Context-Grounded RAG Engine	When a user asks the AI Copilot about their portfolio or a stock, the backend queries the database for user holdings, fetches the latest RSI/MACD indicators, and injects this structured context into the prompt.
Conversational Memory	ChatConversation and ChatMessage models persist conversation history with chronological ordering, allowing multi-turn research dialogues across browser sessions.

Candidate Technical Interview Defense & Viva Strategy

Interview Defense Talking Points (Verbatim Script):

How to defend in an interview: 'Instead of letting the AI assistant hallucinate random stock prices, I designed a RAG pipeline. The assistant queries the live PortfolioAnalyticsService and MarketDataService first. The verified numbers (such as actual realized P&L; and 14-day RSI) are injected into the response generation context. For sentiment, VADER provides deterministic, explainable polarity scores without the latency of heavy cloud LLM calls.'